

PRACTICAL TEST AUTOMATION WITH PLAYWRIGHT

Your First Playwright Test — Walkthrough

Write, understand and run a complete test end to end.

Introduction

This walkthrough takes you through a complete, readable first test so you understand every line — not just copy it. Once this clicks, the rest of automation builds on the same pattern.

The Anatomy of a Test

Part	Role
<code>import { test, expect }</code>	Brings in the test runner and assertions
<code>test('name', async ({ page }) => {...})</code>	Defines a test with a browser page
<code>await page.goto(url)</code>	Navigates to the app
<code>await page.getByRole(...)</code>	Finds an element the way a user would
<code>await expect(...).toBeVisible()</code>	Asserts the expected outcome

Worked Example

A login test: go to the login page, fill email and password using `getByLabel`, click the "Sign in" button using `getByRole`, then assert the dashboard heading is visible. Each step uses `await` because Playwright actions are asynchronous.

The AAA Pattern

- Arrange — navigate and set up state
- Act — perform the user action
- Assert — verify the expected result

Blank Reusable Template

- `test('<clear behaviour>', async ({ page }) => {`

- `// Arrange: await page.goto(...)`
- `// Act: await page.getByRole(...).click()`
- `// Assert: await expect(...).toBeVisible()`
- `});`

Practical Exercise

Write a test for a public demo site: navigate, perform one action, and assert one clear outcome. Run it and view the report.

Best Practices

- Name tests by behaviour, not by steps.
- Always await Playwright actions.
- One clear behaviour per test.

Common Mistakes

- Forgetting `await` (causes flaky/false passes).
- Testing several behaviours in one test.
- Vague test names like `"test1"`.

INSIDE STLC PRO TIP

If you can describe what a test does in one sentence, it is well-scoped. If your sentence needs an "and", consider splitting it into two tests.